Purwadhika Digital Technology School | Digital For Executive

Data Analysis

Session 08: Pandas for Data Analysis

Digital for Executive

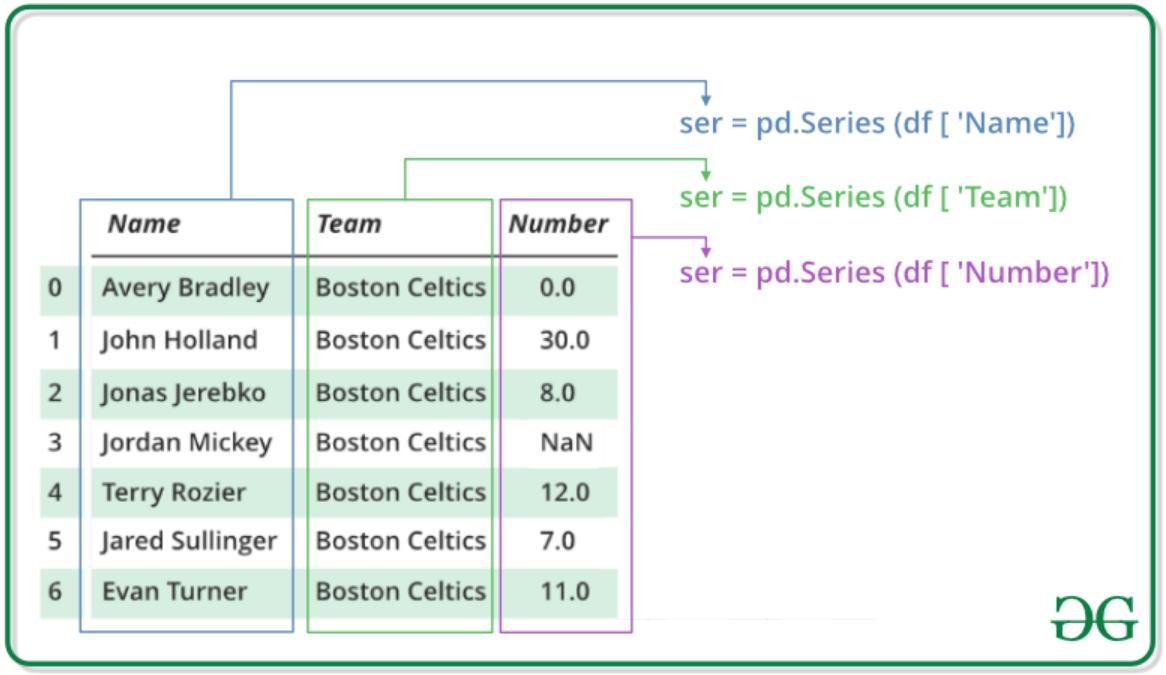
Outline

- Pandas Series & DataFrame
- Pandas Selection & Conditional Selection
- Adding New Data
- Deleting Data
- Pandas Functionality
- Pandas Operation
- Pandas Input & Output

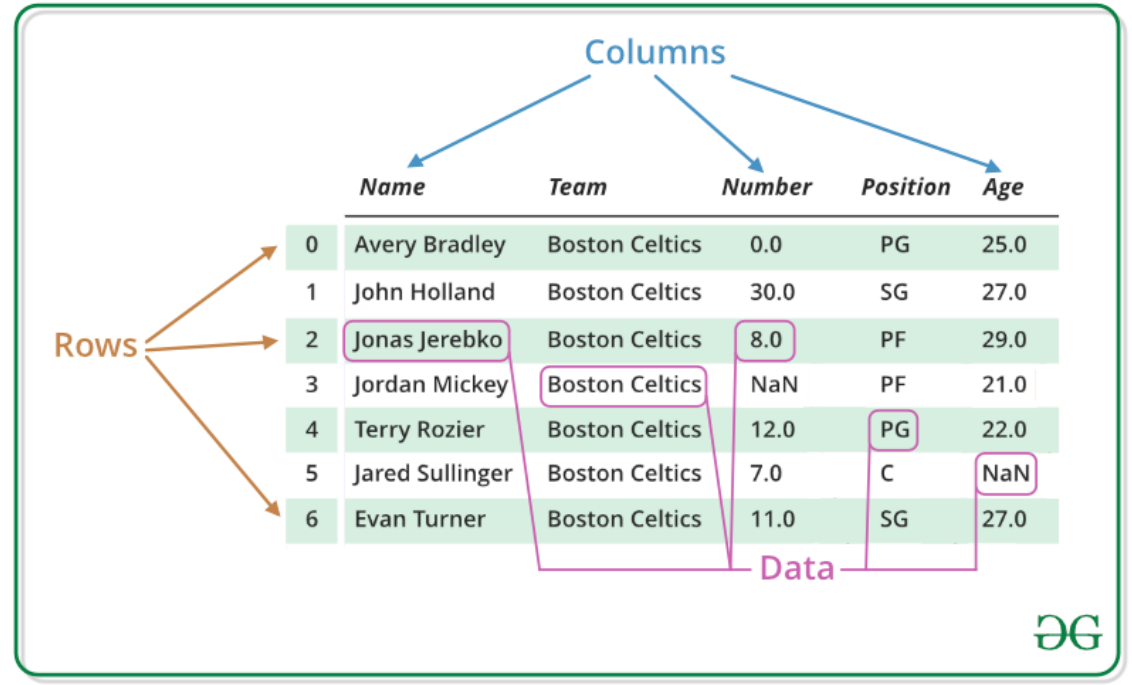
Series and DataFrame

Pandas DataFrame

- A DataFrame is a collection of series with shared index
- It is a structured data: tabular with columns and rows



	Name	Team	Number
0	Avery Bradley	Boston Celtics	0.0
1	John Holland	Boston Celtics	30.0
2	Jonas Jerebko	Boston Celtics	8.0
3	Jordan Mickey	Boston Celtics	NaN
4	Terry Rozier	Boston Celtics	12.0
5	Jared Sullinger	Boston Celtics	7.0
6	Evan Turner	Boston Celtics	11.0



	Name	Team	Number	Position	Age
0	Avery Bradley	Boston Celtics	0.0	PG	25.0
1	John Holland	Boston Celtics	30.0	SG	27.0
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN
6	Evan Turner	Boston Celtics	11.0	SG	27.0

Pandas Series

- A series is a collection of data in 1D array
- Pandas series can hold any type of data (integer, string, float, etc)
- Each data in series can have an axis label, collectively called as index
- A column in a DataFrame is a series

Index label

```
0    10
1    20
2    30
dtype: int32
```

Series

Using Pandas in Python

Import Pandas Library

```
In [1]: import pandas as pd
```

- Before using pandas, first import the pandas library by typing **import pandas as pd.**

Creating Series: from List

- Use **pd.Series()** to create pandas Series

```
In [2]: labels = ['a','b','c']
        my_list = [10, 20, 30]

In [3]: pd.Series(data=my_list)

Out[3]: 0    10
        1    20
        2    30
        dtype: int64
```

```
In [4]: pd.Series(data=my_list, index=labels)

Out[4]: a    10
        b    20
        c    30
        dtype: int64

In [5]: pd.Series(my_list,labels)

Out[5]: a    10
        b    20
        c    30
        dtype: int64
```

Creating Series: from Dictionary

```
In [7]: d= {'a':10, 'b':20, 'c':30}
        pd.Series(d)

Out[7]: a    10
        b    20
        c    30
        dtype: int64
```

- Dictionary keys become index, values become data

Creating DataFrame: from Lists

- Use pd.DataFrame()

```
In [4]: a= [i for i in range (1,5)]
        b= [i for i in range (5,9)]
        c= [i for i in range (9,13)]
```

```
In [5]: pd.DataFrame(data=[a,b,c],
                     index='a b c'.split(),
                     columns= 'w x y z'.split())
```

Out[5]:

	w	x	y	z
a	1	2	3	4
b	5	6	7	8
c	9	10	11	12

- To make each list a column, use zip()

```
In [8]: pd.DataFrame(data=list(zip(a,b,c)),
                     index='w x y z'.split(),
                     columns='a b c'.split())
```

Out[8]:

	a	b	c
w	1	5	9
x	2	6	10
y	3	7	11
z	4	8	12

Creating DataFrame: from a List

- Use transpose (.T)

```
In [7]: my_list= [[1,2,3,4],
                  [5,6,7,8],
                  [9,10,11,12]]
        pd.DataFrame(my_list,
                     index='a b c'.split(),
                      columns= 'w x y z'.split())
```

Out[7]:

	w	x	y	z
a	1	2	3	4
b	5	6	7	8
c	9	10	11	12

```
In [13]: pd.DataFrame(my_list,
                      index='a b c'.split(),
                       columns= 'w x y z'.split()).T
```

Out[13]:

	a	b	c
w	1	5	9
x	2	6	10
y	3	7	11
z	4	8	12

Creating DataFrame: from a Dictionary

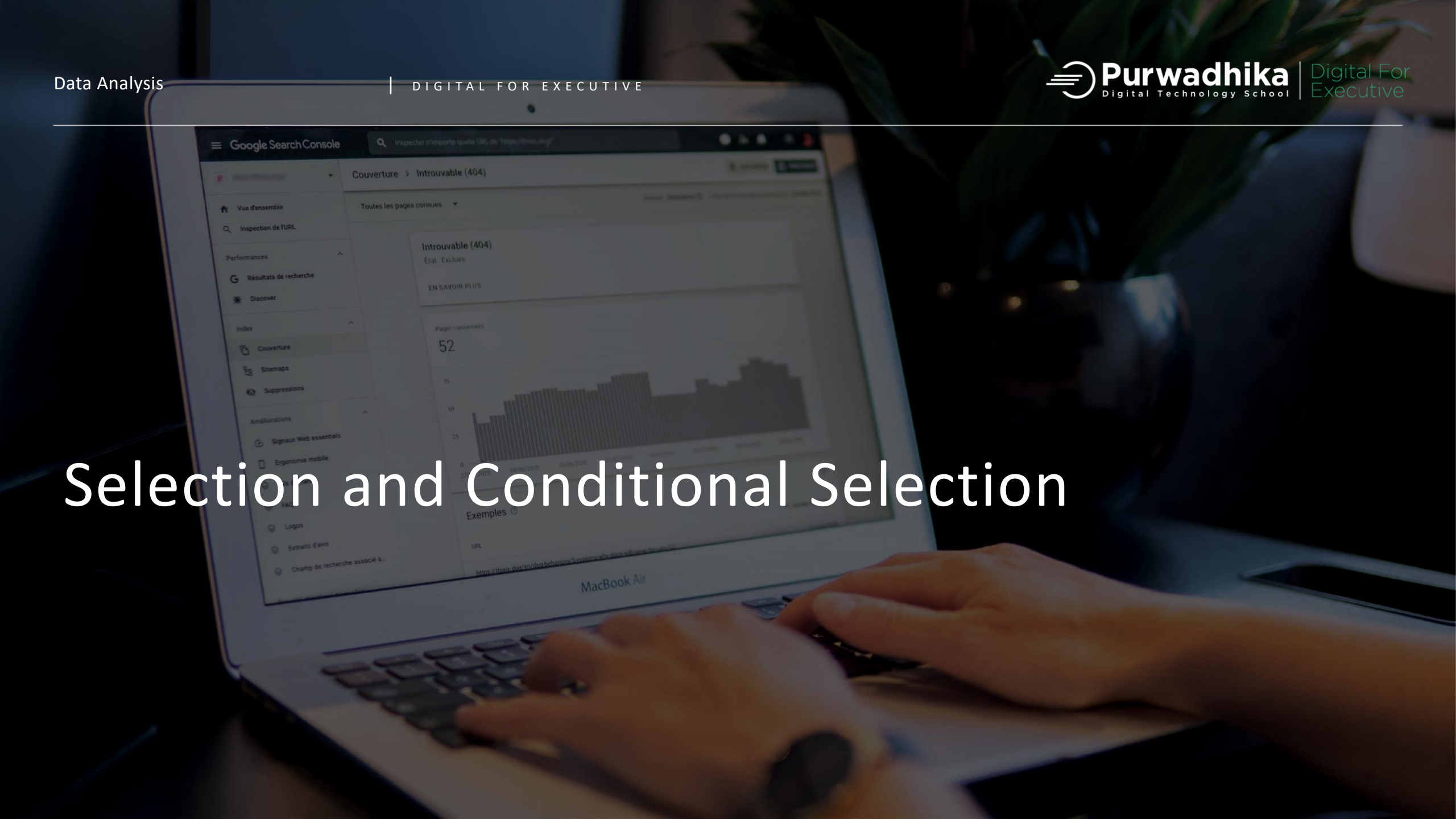
- Each key will be a column

```
In [35]: my_d={'a':[1,2,3,4],
                'b':[5,6,7,8],
                'c':[9,10,11,12]}
         pd.DataFrame(my_d,
                      index= 'w x y z'.split())
```

Out[35]:

	a	b	c
w	1	5	9
x	2	6	10
y	3	7	11
z	4	8	12

- No matter what method, every column has the same length!



Selection and Conditional Selection

Selection & Indexing: The DataFrame

Import Library

```
In [1]: import pandas as pd
```

Create DataFrame

```
In [11]: df= pd.DataFrame({
             'Jakarta':[1,2,3,4,5],
             'Bandung':[2,3,4,5,6],
             'Surabaya':[4,5,6,7,8],
             'Makassar':[5,6,7,8,9]
         }, index= 'Senin Selasa Rabu Kamis Jumat'.split())
```

```
In [12]: df
```

Out[12]:

	Jakarta	Bandung	Surabaya	Makassar
Senin	1	2	4	5
Selasa	2	3	5	6
Rabu	3	4	6	7
Kamis	4	5	7	8
Jumat	5	6	8	9

Selection & Indexing

Selection & Indexing

```
In [13]: df['Jakarta']

Out[13]: Senin     1
         Selasa    2
         Rabu      3
         Kamis     4
         Jumat     5
         Name: Jakarta, dtype: int64

In [14]: df['Jakarta'][0]

Out[14]: 1

In [15]: df['Jakarta'][0:2]

Out[15]: Senin     1
         Selasa    2
         Name: Jakarta, dtype: int64
```

Data type

```
In [16]: type(df['Jakarta'])

Out[16]: pandas.core.series.Series
```

Selection & Indexing

Attribute access, but not recommended

```
In [17]: df.Jakarta

Out[17]: Senin     1
         Selasa    2
         Rabu      3
         Kamis     4
         Jumat     5
         Name: Jakarta, dtype: int64
```

Selection & Indexing

Attribute access, but not recommended

In [18]:
```
df['min']=[1,2,3,4,5]
df
```

Out[18]:

	Jakarta	Bandung	Surabaya	Makassar	min
Senin	1	2	4	5	1
Selasa	2	3	5	6	2
Rabu	3	4	6	7	3
Kamis	4	5	7	8	4
Jumat	5	6	8	9	5

In [19]:
```
df.min
```

Out[19]:
```
<bound method DataFrame.min of          Jakarta  Bandung  Surabaya  Makassar  min
Senin         1        2         4         5    1
Selasa        2        3         5         6    2
Rabu          3        4         6         7    3
Kamis         4        5         7         8    4
Jumat         5        6         8         9    5>
```

Selection & Indexing

Recommended

```
In [20]: df['min']

Out[20]: Senin     1
         Selasa    2
         Rabu      3
         Kamis     4
         Jumat     5
         Name: min, dtype: int64
```

More Columns

```
In [21]: df[['Jakarta','Surabaya']]

Out[21]:
```

	Jakarta	Surabaya
Senin	1	4
Selasa	2	5
Rabu	3	6
Kamis	4	7
Jumat	5	8

Selection & Indexing: loc

DataFrame df

	Jakarta	Bandung	Surabaya	Makassar
Senin	1	2	4	5
Selasa	2	3	5	6
Rabu	3	4	6	7
Kamis	4	5	7	8
Jumat	5	6	8	9

```
In [26]: df.loc['Senin']

Out[26]: Jakarta     1
         Bandung     2
         Surabaya    4
         Makassar    5
         Name: Senin, dtype: int64

In [27]: df.loc['Senin':'Rabu']
```

Out[27]:

	Jakarta	Bandung	Surabaya	Makassar
Senin	1	2	4	5
Selasa	2	3	5	6
Rabu	3	4	6	7

Selection & Indexing: iloc

DataFrame df

	Jakarta	Bandung	Surabaya	Makassar
Senin	1	2	4	5
Selasa	2	3	5	6
Rabu	3	4	6	7
Kamis	4	5	7	8
Jumat	5	6	8	9

```
In [28]: df.iloc[2]

Out[28]: Jakarta     3
         Bandung     4
         Surabaya    6
         Makassar    7
         Name: Rabu, dtype: int64

In [29]: df.iloc[0:4:2]
```

Out[29]:

	Jakarta	Bandung	Surabaya	Makassar
Senin	1	2	4	5
Rabu	3	4	6	7

Selection & Indexing: iloc

DataFrame df

	Jakarta	Bandung	Surabaya	Makassar
Senin	1	2	4	5
Selasa	2	3	5	6
Rabu	3	4	6	7
Kamis	4	5	7	8
Jumat	5	6	8	9

```
In [30]: df.iloc[:,0:2]
Out[30]:
```

	Jakarta	Bandung
Senin	1	2
Selasa	2	3
Rabu	3	4
Kamis	4	5
Jumat	5	6

```
In [31]: df.iloc[1,3]
Out[31]: 6

In [32]: df.iloc[[1,3]]
Out[32]:
```

	Jakarta	Bandung	Surabaya	Makassar
Selasa	2	3	5	6
Kamis	4	5	7	8

Conditional Selection

DataFrame df

	Jakarta	Bandung	Surabaya	Makassar
Senin	1	2	4	5
Selasa	2	3	5	6
Rabu	3	4	6	7
Kamis	4	5	7	8
Jumat	5	6	8	9

```
In [33]: df>5
```

Out[33]:

	Jakarta	Bandung	Surabaya	Makassar
Senin	False	False	False	False
Selasa	False	False	False	True
Rabu	False	False	True	True
Kamis	False	False	True	True
Jumat	False	True	True	True

Conditional Selection

DataFrame df

	Jakarta	Bandung	Surabaya	Makassar
Senin	1	2	4	5
Selasa	2	3	5	6
Rabu	3	4	6	7
Kamis	4	5	7	8
Jumat	5	6	8	9

```
In [34]: df[df>5]
```

Out[34]:

	Jakarta	Bandung	Surabaya	Makassar
Senin	NaN	NaN	NaN	NaN
Selasa	NaN	NaN	NaN	6.0
Rabu	NaN	NaN	6.0	7.0
Kamis	NaN	NaN	7.0	8.0
Jumat	NaN	6.0	8.0	9.0

```
In [37]: df[df['Surabaya']>5]
```

Out[37]:

	Jakarta	Bandung	Surabaya	Makassar
Rabu	3	4	6	7
Kamis	4	5	7	8
Jumat	5	6	8	9

```
In [38]: df[df['Surabaya']>5]['Surabaya']
```

```
Out[38]: Rabu     6
         Kamis    7
         Jumat    8
         Name: Surabaya, dtype: int64
```

Conditional Selection

DataFrame df

	Jakarta	Bandung	Surabaya	Makassar
Senin	1	2	4	5
Selasa	2	3	5	6
Rabu	3	4	6	7
Kamis	4	5	7	8
Jumat	5	6	8	9

```
In [39]: df[df['Surabaya']>5][['Surabaya','Makassar']]
```

Out[39]:

	Surabaya	Makassar
Rabu	6	7
Kamis	7	8
Jumat	8	9

```
In [41]: df[(df['Surabaya']>5) & (df['Jakarta']< 4)]
```

Out[41]:

	Jakarta	Bandung	Surabaya	Makassar
Rabu	3	4	6	7

Conditional Selection

```
df
```

	name	gender	hire date	gross salary
100111	Raven Bierman	Female	2016-12-04	7000000
100112	Valter Havers	Male	2018-04-13	7000000
200210	Marko Mendell	Male	2018-07-04	15000000
200211	Takahiro Momota	Male	2016-11-18	12000000
200312	Yahiko Tilemans	Male	2017-05-26	20000000
300207	Dina Rebaine	Female	2015-03-20	15000000

```
df[df['name'] == 'Raven Bierman']
```

	name	gender	hire date	gross salary
100111	Raven Bierman	Female	2016-12-04	7000000

```
df[df['gender'] == 'Male']
```

	name	gender	hire date	gross salary
100112	Valter Havers	Male	2018-04-13	7000000
200210	Marko Mendell	Male	2018-07-04	15000000
200211	Takahiro Momota	Male	2016-11-18	12000000
200312	Yahiko Tilemans	Male	2017-05-26	20000000

Adding and Deleting Data

Adding and Deleting Data

DataFrame df

	Jakarta	Bandung	Surabaya	Makassar
Senin	1	2	4	5
Selasa	2	3	5	6
Rabu	3	4	6	7
Kamis	4	5	7	8
Jumat	5	6	8	9

Adding a New Row

```
df.loc['new']=[10,11,12,13]
df
```

	Jakarta	Bandung	Surabaya	Makassar
Senin	1	2	4	5
Selasa	2	3	5	6
Rabu	3	4	6	7
Kamis	4	5	7	8
Jumat	5	6	8	9
new	10	11	12	13

Adding a New Column

```
df['new']=df['Jakarta']+df['Bandung']
df
```

	Jakarta	Bandung	Surabaya	Makassar	new
Senin	1	2	4	5	3
Selasa	2	3	5	6	5
Rabu	3	4	6	7	7
Kamis	4	5	7	8	9
Jumat	5	6	8	9	11

Adding a New Column: Insert

```
df.insert(2,'new',[1,2,3,4,5])
df
```

	Jakarta	Bandung	new	Surabaya	Makassar
Senin	1	2	1	4	5
Selasa	2	3	2	5	6
Rabu	3	4	3	6	7
Kamis	4	5	4	7	8
Jumat	5	6	5	8	9

Deleting a Column:

Without inplace

	Jakarta	Bandung	new	Surabaya	Makassar
Senin	1	2	1	4	5
Selasa	2	3	2	5	6
Rabu	3	4	3	6	7
Kamis	4	5	4	7	8
Jumat	5	6	5	8	9

```
df.drop('new',axis=1)
```

	Jakarta	Bandung	Surabaya	Makassar
Senin	1	2	4	5
Selasa	2	3	5	6
Rabu	3	4	6	7
Kamis	4	5	7	8
Jumat	5	6	8	9

```
# Not inplace unless specified!
df
```

	Jakarta	Bandung	new	Surabaya	Makassar
Senin	1	2	1	4	5
Selasa	2	3	2	5	6
Rabu	3	4	3	6	7
Kamis	4	5	4	7	8
Jumat	5	6	5	8	9

Deleting a Column: With inplace

```
df.drop('new',axis=1,inplace=True)
df
```

	Jakarta	Bandung	Surabaya	Makassar
Senin	1	2	4	5
Selasa	2	3	5	6
Rabu	3	4	6	7
Kamis	4	5	7	8
Jumat	5	6	8	9

Deleting a Row

Same with drop columns, the difference is the axis:

```
df.drop('Rabu',axis=0,inplace=True)
df
```

	Jakarta	Bandung	Surabaya	Makassar
Senin	1	2	4	5
Selasa	2	3	5	6
Kamis	4	5	7	8
Jumat	5	6	8	9

Pandas Functionality

Functionality: head & tail

```
df.head()
```

	Id	EmployeeName	JobTitle	BasePay	OvertimePay	OtherPay	Benefits	TotalPay	TotalPayBenefits	Year	Notes	Agency	Status
0	1	NATHANIEL FORD	GENERAL MANAGER-METROPOLITAN TRANSIT AUTHORITY	167411.18	0.00	400184.25	NaN	567595.43	567595.43	2011	NaN	San Francisco	NaN
1	2	GARY JIMENEZ	CAPTAIN III (POLICE DEPARTMENT)	155966.02	245131.88	137811.38	NaN	538909.28	538909.28	2011	NaN	San Francisco	NaN
2	3	ALBERT PARDINI	CAPTAIN III (POLICE DEPARTMENT)	212739.13	106088.18	16452.60	NaN	335279.91	335279.91	2011	NaN	San Francisco	NaN
3	4	CHRISTOPHER CHONG	WIRE ROPE CABLE MAINTENANCE MECHANIC	77916.00	56120.71	198306.90	NaN	332343.61	332343.61	2011	NaN	San Francisco	NaN
4	5	PATRICK GARDNER	DEPUTY CHIEF OF DEPARTMENT,(FIRE DEPARTMENT)	134401.60	9737.00	182234.59	NaN	326373.19	326373.19	2011	NaN	San Francisco	NaN

```
df.tail()
```

	Id	EmployeeName	JobTitle	BasePay	OvertimePay	OtherPay	Benefits	TotalPay	TotalPayBenefits	Year	Notes	Agency	Status
148649	148650	Roy I Tillery	Custodian	0.0	0.0	0.00	0.0	0.00	0.00	2014	NaN	San Francisco	NaN
148650	148651	Not provided	Not provided	NaN	NaN	NaN	NaN	0.00	0.00	2014	NaN	San Francisco	NaN
148651	148652	Not provided	Not provided	NaN	NaN	NaN	NaN	0.00	0.00	2014	NaN	San Francisco	NaN
148652	148653	Not provided	Not provided	NaN	NaN	NaN	NaN	0.00	0.00	2014	NaN	San Francisco	NaN
148653	148654	Joe Lopez	Counselor, Log Cabin Ranch	0.0	0.0	-618.13	0.0	-618.13	-618.13	2014	NaN	San Francisco	NaN

Functionality: info

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 148654 entries, 0 to 148653
Data columns (total 13 columns):
Id                  148654 non-null int64
EmployeeName        148654 non-null object
JobTitle            148654 non-null object
BasePay             148045 non-null float64
OvertimePay         148650 non-null float64
OtherPay            148650 non-null float64
Benefits            112491 non-null float64
TotalPay            148654 non-null float64
TotalPayBenefits    148654 non-null float64
Year                148654 non-null int64
Notes               0 non-null float64
Agency              148654 non-null object
Status              0 non-null float64
dtypes: float64(8), int64(2), object(3)
memory usage: 14.7+ MB
```

Pandas Operation

Pandas Operation

The DataFrame

In [1]:
```
import pandas as pd
df = pd.DataFrame({'col1':[1,2,3,4],'col2':[444,555,666,444],'col3':['abc','def','ghi','xyz']})
df.head()
```

Out[1]:

	col1	col2	col3
0	1	444	abc
1	2	555	def
2	3	666	ghi
3	4	444	xyz

Pandas Operation: Apply

```
In [7]: def times2(x):
            return x*2

In [8]: df['col1'].apply(times2)

Out[8]: 0    2
        1    4
        2    6
        3    8
        Name: col1, dtype: int64
```

```
In [9]: df['col3'].apply(len)

Out[9]: 0    3
        1    3
        2    3
        3    3
        Name: col3, dtype: int64
```

Pandas Operation: Apply with Lambda

```
In [11]: df['col1'].apply(lambda x: x*2)

Out[11]: 0    2
         1    4
         2    6
         3    8
         Name: col1, dtype: int64

In [12]: df['col3'].apply(lambda x: x[1])

Out[12]: 0    b
         1    e
         2    h
         3    y
         Name: col3, dtype: object

In [13]: df['col3'].apply(lambda x: len(x))

Out[13]: 0    3
         1    3
         2    3
         3    3
         Name: col3, dtype: int64
```

Input dan Output

Input & Ouput

Input: csv

```
In [2]: df=pd.read.csv('example.csv')
        df
```

Out[2]:

	a	b	c	d
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11
3	12	13	14	15

Output: csv

```
In [3]: pd.to_csv('example.csv',index=False)
```

Input: Excel

```
In [4]: pd.read_excel('Excel_Sample.xlsx',sheet_name='Sheet1')
```

Out[4]:

	a	b	c	d
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11
3	12	13	14	15

Output: Excel

```
In [5]: df.to_excel('Excel_Sample.xlsx',sheet_name='Sheet1')
```

Input: JSON

‘records’ writing structure

```
[{"nama":"Andi","umur":25,"pekerjaan":"Front End Developer"},
    {"nama":"Budi","umur":18,"pekerjaan":"Back End Developer"},
    {"nama":"Caca","umur":36,"pekerjaan":"Mobile Developer"}]
```

```
df = pd.read_json('data.json', orient='records')
df.head()
```

executed in 14ms, finished 14:23:04 2019-12-08

	nama	pekerjaan	umur
0	Andi	Front-End Developer	24
1	Budi	Back-End Developer	18
2	Caca	Mobile Developer	30

Output: JSON

```
df = pd.DataFrame({
    'nama': ['Andi','Budi','Caca'],
    'umur': [24,18,30],
    'pekerjaan': ['Front-End Developer','Back-End Developer', 'Mobile Developer']
})
df.head()
```

executed in 21ms, finished 14:18:55 2019-12-08

	nama	umur	pekerjaan
0	Andi	24	Front-End Developer
1	Budi	18	Back-End Developer
2	Caca	30	Mobile Developer

```
df.to_json('data.json',orient='records')
```

executed in 4ms, finished 14:20:31 2019-12-08

Input: HTML

```
In [6]: df = pd.read_html('http://www.fdic.gov/bank/individual/failed/banklist.html')
```

```
In [8]: df[0].head()
```

Out[8]:

	Bank Name	City	ST	CERT	Acquiring Institution	Closing Date	Updated Date
0	Washington Federal Bank for Savings	Chicago	IL	30570	Royal Savings Bank	December 15, 2017	February 21, 2018
1	The Farmers and Merchants State Bank of Argonia	Argonia	KS	17719	Conway Bank	October 13, 2017	February 21, 2018
2	Fayette County Bank	Saint Elmo	IL	1802	United Fidelity Bank, fsb	May 26, 2017	July 26, 2017
3	Guaranty Bank, (d/b/a BestBank in Georgia & Mi...	Milwaukee	WI	30003	First-Citizens Bank & Trust Company	May 5, 2017	March 22, 2018
4	First NBC Bank	New Orleans	LA	58302	Whitney Bank	April 28, 2017	December 5, 2017

For More Information

Check Pandas documentation here

Thank You!

PURWADHIKA HEAD OFFICE

BSD Green Office Park 9 - G Floor,
BSD City, Tangerang, Banten

CAMPUS BRANCHES

Purwadhika BSD
Purwadhika Jakarta
Purwadhika Bandung

CONTACT

021 - 5012 3161
0813 - 1867 8671
0812 - 8060 7160

EMAIL

admission1@purwadhika.com
admission2@purwadhika.com
admission3@purwadhika.com

WEBSITE

www.purwadhika.com